

Università degli Studi di Salerno

Corso di Ingegneria del Software

**Easy Work Management System
Object Design Document
Versione 1.3**



E.W.M.S.

Easy Work Management System

Data: 09/02/2026

Progetto: Easy Work Management System	Versione: 1.3
Documento: Object Design Document	Data: 09/02/2026

Coordinatore del progetto:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Partecipanti:

Nome	Matricola
Giovanni Robustelli	0512121282
Vincenzo Mauro	0512119080

Scritto da:	Vincenzo Mauro, Giovanni Robustelli
--------------------	-------------------------------------

Revision History

Data	Versione	Descrizione	Autore
15/12/2025	1.0	Assemblaggio delle singole parti sviluppate dai partecipanti	Giovanni Robustelli Vincenzo Mauro
22/01/2026	1.1	Modificate e aggiunti nuovi metodi per UtenteDAO e TaskDAO	Giovanni Robustelli
24/01/2026	1.2	Aggiunti nuovi metodi per UtenteDAO	Giovanni Robustelli
26/01/2026	1.2.1	Modificati metodi per servizi di AccountManagement e AccessManagement	Giovanni Robustelli
30/01/2026	1.2.2	Modificati e aggiunti metodi per tutti i sottosistemi dell'application layer, aggiornati i metodi in modo da prevedere il cambiamento di rappresentazione della matricola di un utente e aggiunti metodi necessari per la visualizzazione delle informazioni sia di task che degli account	Giovanni Robustelli
09/02/2026	1.3	Ultima revisione da parte del team	Giovanni Robustelli Vincenzo Mauro

Progetto: Easy Work Management System	Versione: 1.3
Documento: Object Design Document	Data: 09/02/2026

--	--	--	--

Indice

1.	Introduzione	5
1.1	Object design trade-offs	5
1.2	Interface documentation guidelines	6
2.	Pacchetti	7
2.1	Decomposizione dei sottosistemi in package	8
2.3	Dipendenze tra package.....	10
3.	Class interfaces	10
3.1	AccessManagement.....	10
3.2	AccountManagement	11
3.3	TaskManagement	16
3.4	NotificheManagement.....	22
3.5	PersistenceManagement	23
	Classi DAO	24

1. Introduzione

E.W.M.S. è una piattaforma di gestione dei task progettata per facilitare la comunicazione tra dipendenti e supervisori in contesti operativi caratterizzati da un elevato volume di attività, come magazzini, centri logistici e reparti industriali. Il sistema punta a centralizzare l'assegnazione dei compiti e il monitoraggio dello stato delle attività, riducendo i tempi di coordinamento e aumentando l'efficienza e la trasparenza organizzativa.

Questo documento serve a chiarire e presentare le decisioni riguardanti l'object design su cui ci si baserà per costruire il prodotto E.W.M.S., oltre che a definire la decomposizione del sistema in package e le loro responsabilità, il mapping tra requisiti funzionali e oggetti di design, nonché la relazione tra questi ultimi e i pattern adottati.

Inoltre, verranno mostrate le strategie di persistenza e gli aspetti che determinano la qualità del design, tutto questo per favorire l'implementazione e lo sviluppo del sistema.

1.1 Object design trade-offs

Complessità vs. manutenibilità

È nel nostro interesse utilizzare un modello ad oggetti semplice e composto da poche classi centrali suddivise in maniera non eccessiva, riducendo il rischio di over-design e accelerando il processo di sviluppo.

Questa semplificazione tende a ridurre la capacità del modello di rappresentare esaustivamente delle operazioni/situazioni specifiche che però non si applica sullo scope del nostro prodotto, poiché sono previste astrazioni tali da coprire la maggior parte degli aspetti richiesti da un classico task manager.

Questo si traduce in una strutturazione del codice facile da applicare e comprendere, in modo da adattare quanto più possibile il prodotto ai design goals specificati nel SDD.

Per quanto riguarda la manutenzione del sistema, se ne occuperà il team di sviluppo originale (vedi RAD, 3.3.4 Supportabilità – Manutenibilità), quindi all'azienda che adotterà il prodotto non verrà richiesto impegno aggiuntivo.

Import vs. build

Tutti i componenti principali del sistema saranno implementati internamente, assicurando il controllo completo sulla logica di business, saranno invece adottate librerie standard esterne per le funzionalità infrastrutturali, riducendo i costi di sviluppo e i rischi di errori da risolvere con la manutenzione.

Memoria vs. performance

Il sistema si basa su strutture dati semplici con un caricamento dei dati “on-demand”, favorendo i tempi di risposta rispetto ad una memorizzazione temporanea prolungata, e quindi le performance.

Questa scelta minimizza il consumo di memoria e aumenta la flessibilità del sistema a fronte di molteplici richieste di servizio e interrogazione al database.

Ottimizzazione vs. usabilità

Il sistema è ideato per essere user-friendly, in modo da favorire l’accessibilità e l’usabilità dello stesso, senza però sacrificare gli aspetti tecnici come performance, scalabilità e affidabilità.

Inoltre, utilizzando interfacce generiche, vengono privilegiate anche l’estendibilità e la sostituibilità delle componenti del sistema.

Tuttavia, questa scelta limita le prestazioni ottenute rispetto ad un’implementazione focalizzata sulla performance.

1.2 Interface documentation guidelines

Al fine di garantire coerenza, qualità e comprensibilità del progetto, l’object design segue le seguenti linee guida:

Object naming conventions:

- le **classi** utilizzeranno nomi abbastanza descrittivi della loro funzione e seguiranno il metodo di scrittura *PascalCase*.
- I **metodi** seguiranno la stessa metodica delle classi, descrivendo un’azione nel loro nome, scritto in *camelCase*.
- Gli **attributi** rappresenteranno proprietà dell’oggetto, utilizzando sostantivi brevi e significativi.

- Ogni **package** rappresenterà un ambito funzionale del sistema (Task, Notifiche, Repository)

Boundary Cases:

verrà riposta particolare attenzione nei casi limite, come input nulli, inconsistenza degli oggetti, task non assegnati o già completati, connessioni interrotte.

Naturalmente le classi prevedono e definiscono comportamenti prevedibili e documentati per questi casi.

Exception Handling:

le eccezioni sono previste come meccanismo di controllo del flusso della fruibilità del sistema e come segnalazioni di condizioni anomale, che potranno essere risolte o comprese anche tramite consulenza al team di sviluppo.

Le eccezioni saranno inoltre suddivise in eccezioni causate da errori di validazione, comunicazione e persistenza.

2. Pacchetti

Questa sezione descrive la decomposizione dei subsystems in package software e l'organizzazione dei file del codice sorgente per il sistema *Easy Work Management System* (E.W.M.S.).

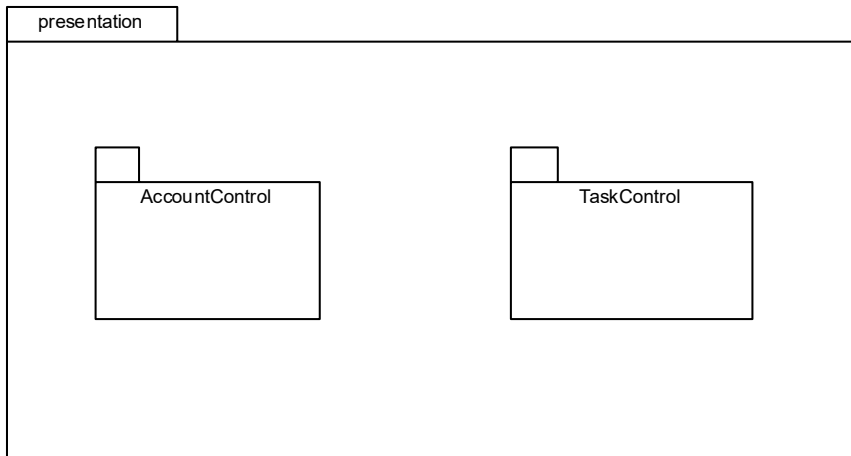
L'organizzazione dei package segue l'architettura Three-Tier definita nel System Design Document, separando chiaramente la logica di presentazione, la logica di business e la persistenza dei dati. Il codice sorgente Java è organizzato sotto il root package **it.unisa.ewms**. La struttura riflette la suddivisione logica in layer e l'adozione del pattern Model-View-Controller (MVC).

I principali gruppi di package identificati sono:

- Presentation package
- Application package
- Model package
- Persistence package

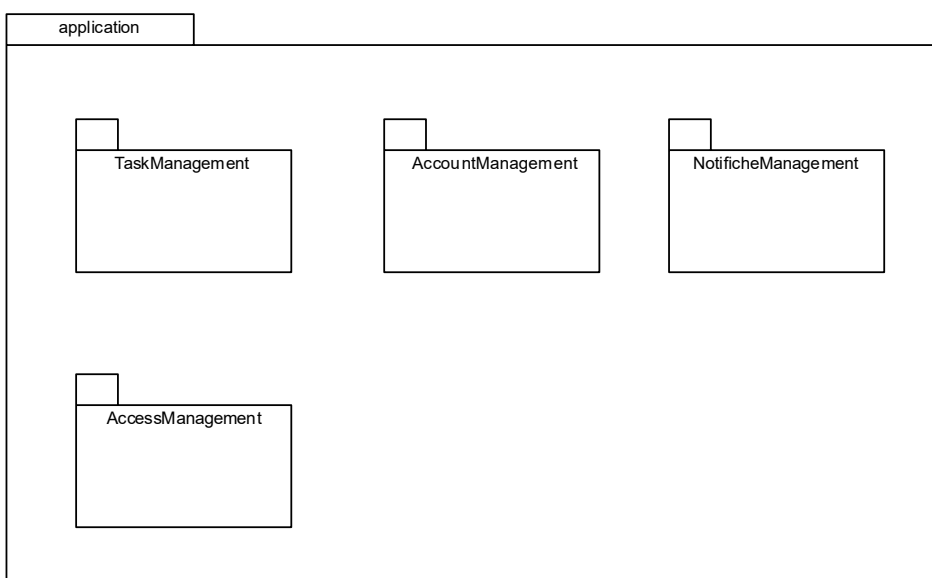
2.1 Decomposizione dei sottosistemi in package

it.unisa.ewms.presentation



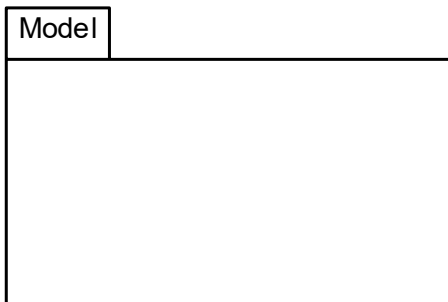
Questo package implementa il Presentation Layer e corrisponde ai componenti, AccountControl e TaskControl. Include le classi Servlet che fungono da Controller (nei sottopacchetti AccountControl e TaskControl).

it.unisa.ewms.application



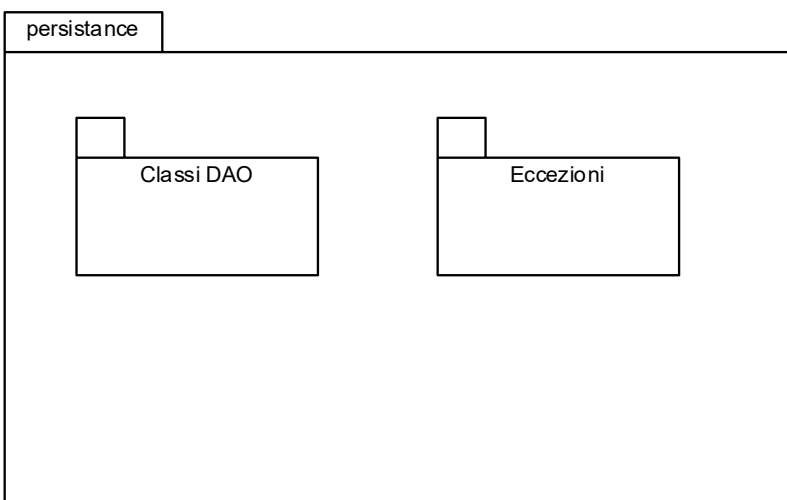
Questo package implementa l'Application Layer e raggruppa i servizi di business logic AccessManagement, TaskManagement, NotificheManagement e AccountManagement, ognuno dei quali è racchiuso nel corrispondente sottopacchetto.

it.unisa.ewms.model



Questo package rappresenta i dati trasversali a tutti i livelli e corrisponde alle entità definite nel RAD. Contiene quindi i bean che verranno utilizzati come mezzo di trasporto delle informazioni.

it.unisa.ewms.persistence



Questo package implementa il Persistence Layer e il componente PersistenceManagement. Implementa il pattern DAO (Data Access Object). Gestisce la connessione al DBMS MySQL tramite JDBC, l'esecuzione delle query SQL e il mapping dei ResultSet in oggetti del Model.

2.3 Dipendenze tra package

Le dipendenze tra i package sono regolate rigorosamente per evitare dipendenze cicliche e rispettare la separazione dei compiti.

- Flusso di Dipendenza:
 1. *presentation* dipende da *application* e *model*. I Controller invocano i metodi dei servizi per eseguire operazioni.
 2. *application* dipende da *persistence* e *model*. I servizi coordinano le chiamate ai DAO per recuperare o salvare dati.
 3. *persistence* dipende solo da *model* e dalle librerie JDBC. Non ha conoscenza della logica di business o della presentazione.
 4. *model* è indipendente e non ha dipendenze verso gli altri package del sistema.

3. Class interfaces

Questa sezione del documento descrive in dettaglio le Class interfaces progettate per il sistema Easy Work Management System (E.W.M.S.). L'obiettivo è fornire una specifica tecnica completa delle componenti software, definendo i contratti pubblici che regolano le interazioni tra gli oggetti del sistema.

3.1 AccessManagement

3.1.1 SessionService

Nome classe	SessionService
Descrizione	Classe di servizio che incapsula la logica di business relativa all'autenticazione e all'autorizzazione. Interagisce con il persistence layer (tramite CatalogoUtenti) per la verifica delle credenziali e gestisce lo stato della HttpSession.
Metodi	+ login(email: String, password: String): Boolean + getUtente(email:String): Utente
Invariante	service:PersistenceService <> null

Firma metodo	+ login(email: String, password: String): Boolean
Descrizione	Verifica la corrispondenza delle credenziali fornite.
Pre-condizione	Context SessionService::login username $\neq$ null AND password $\neq$ null.
Post-condizione	Context SessionService::login If Utente.allInstances()->exists(u u.email = email and u.hashpassword = password) result = true else result = false
Firma metodo	+ getUtente(email:String): Utente
Descrizione	Ritorna un'istanza di Utente per permettere la corretta inizializzazione della sessione.
Pre-condizione	Context:SessionService::getUtente Email $\neq$ null
Post-condizione	Context SessionService::getUtente If Utente.getIstances()-> exists(u u.email = email) result = Utente.getIstances()-> select(u u.email = email) else result = null

3.2 AccountManagement

Per chiarezza del documento all'interno delle precondizioni per i metodi:

- addAccount

Vale anche la seguente precondizione:

utente:Utente

utente.email $\neq$ null and

utente.email.matches(^[a-zA-Z0-9]{1,50}\.[a-zA-Z0-9]{1,50}@azienda\.it\$) and

utente.nome $\neq$ null and

utente.nome.size() > 0 and utente.nome.size() <= 50 and
 utente.cognome <> null and
 utente.cognome.size() > 0 and utente.cognome.size() <= 50 and
 utente.ruolo <> null and
 utente.dataNasc <> null

Inoltre, si definisce

formatoPassword = $^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\!%*?&])[A-Za-z\d@!\%*?&]{8,16}\$$

3.2.1 *PersonaProfileService*

Nome Classe	PersonalProfileService
Descrizione	gestisce tutte le operazioni che un utente autenticato può eseguire sul proprio account
Metodi	+ changePwd(utente:Utente, oldpwd: string, newpwd: string) : bool
Invariante di classe	service:PersistenceService <> null
Nome Metodo	+ changePwd(utente:Utente, oldpwd: string, newpwd: string) : bool
Descrizione	Permette all'utente di aggiornare la propria password.
Pre-condizione	Context PersonalProfileService::changePwd(utente:Utente, oldpwd: string, newpwd: string) utente <> null and utente.matricola > 0 and oldpwd <> null and oldpwd <> "" and newpwd <> null and newpwd <> ""
Post-condizione	Context PersonalProfileService::changePwd(utente:Utente, oldpwd: string, newpwd: string) targetUser = Utente.allInstances()->any(u u.matricola = utente.matricola) targetUser.hashPassword = newpwd

3.2.2 ProfileManagementService

Nome Classe	ProfileManagementService
Descrizione	gestisce le operazioni amministrative riservate al ruolo del Gestore degli account
Metodi	+ addAccount(utente:Utente, password:String) : void + getAccount(matricola:int) : Utente + getAllAccount(): List<Utente> +getAllSupervisor(): List<Informazione> + modifyRole(utente:Utente, newRole:String, supMatricola:String) : void + modifySupervisor(utente:Utente, supMatricola:int) : void + replacePassword(matricola:int, newPwd:String) : void + deleteAccount(matricola:int) : void
Invariante di classe	service:PersistenceService <> null
Firma Metodo	+ addAccount(utente:Utente, password:String) : void
Descrizione	Permette di aggiungere un nuovo account al sistema
Pre-condizione	Context ProfileManagementService::addAccount(utente: Utente) utente <> null and password <> null and password.matches(formatoPassword)
Post-condizione	Context ProfileManagementService result = getAccount(u.matricola) result <> null
Firma Metodo	+ getAccount(matricola:int) : Utente
Descrizione	Permette di recuperare i dati di un account selezionato dal client.
Pre-condizione	Context ProfileManagementService::getAccount(matricola:String) matricola > 0
Post-condizione	Context ProfileManagementService::getAccount(matricola:String) utenteTarget = Utente.allInstances()->any(u u.matricola = matricola)

	<pre> if utenteTarget <> null then result = utenteTarget else result = null endif </pre>
Firma Metodo	+ getAllAccounts() : List<Utente >
Descrizione	Recupera dal database tutti gli account attualmente registrati all'interno del database
Pre-condizione	/
Post-condizione	Context ProfileManagementService::getAllAccount() Result = Utente.AllInstances()-> select()
Firma Metodo	+getAllSupervisor(): List<Informazione>
Descrizione	Recupera le informazioni fondamentali (matricola, nome, cognome) per tutti i supervisor presenti all'interno del sistema
Pre-condizione	/
Post-condizione	Context ProfileManagementService::getAllSupervisor() Result = Supervisore.allInstances()-> select()
Firma Metodo	+ modifyRole(utente: Utente, newRole: enum, matricolaSup:String) : void
Descrizione	Permette di cambiare il ruolo assegnato ad un account.
Pre-condizione	Context ProfileManagementService::modifyRole(utente: Utente, newRole:enum) utente <> null and newRole <> null and utente.matricola > 0 and (newRole = Tipi::ruolo::DIPENDENTE) implies (matricolaSup > 0 and matricolaSup <> utente.matricola
Post-condizione	Context ProfileManagementService::modifyRole(utente: Utente, newRole:enum) target = Utente.allInstances()->any(u u.matricola = utente.matricola) target.ruolo = newRole and if newRole = Tipi::ruolo::DIPENDENTE then Dipendente.allInstances()->exists(d d.matricola = target.matricola and d.supMatricola = matricolaSup) else if newRole = Tipi::ruolo::SUPERVISORE then Supervisore.allInstances()->exists(s s.matricola = target.matricola)

	<pre> else Utente.allInstances()->exists(s s.matricola = target.matricola) endif endif else target.ruolo = target.ruolo@pre endif </pre>
Firma Metodo	+ modifySupervisor(utente:Utente, supMatricola:int) : void
Descrizione	Permette di cambiare supervisore ad un dipendente.
Pre-condizione	Context ProfileManagementService::modifySupervisor(utente:Utente, supMatricola:int) Utente <> null and utente.ruolo = Tipi::ruolo::DIPENDENTE and utente.matricola > 0 and supMatricola > 0 and utente.matricola <> supMatricola
Post-condizione	Context ProfileManagementService::modifySupervisor(utente:Utente, supMatricola:int) targetDipendente = ProfileManagementService.getAccount(utente.matricola) if targetDipendente <> null then targetDipendente.supMatricola = supMatricola else targetDipendente.supMatricola = targetDipendente.supMatricola@pre endif
Firma Metodo	+ replacePassword(matricola:int, newPwd:String) : void
Descrizione	Permette di rimpiazzare la password di un account all'interno del database
Pre-condizione	Context replacePassword(matricola:String, newPwd:String) matricola > 0 and newPwd <> null and newPwd.matches((^(?=.*[a-z])(?=.*[A-Z])(?=.*\d)(?=.*[@\$!%*?&])[A-Za-z\d@\$!%*?&]{8,16}\$)
Post-condizione	Context replacePassword(matricola:int, newPwd:String) targetPassword = Utente.allInstances()->any(u u.matricola = matricola) if result = true then targetUtente.hashPassword = newPwd else

	targetUtente.hashPassword = targetUtente.hashPassword@pre endif
Firma Metodo	+ deleteAccount(utente: Utente) : void
Descrizione	Permette di eliminare un account dal database
Pre-condizione	Context ProfileManagementService:: deleteAccount(utente:Utente) Utente <> null and utente.matricola > 0
Post-condizione	Context ProfileManagementService:: deleteAccount(utente:Utente) not Utente.allInstances()->exists(u u.matricola = account.matricola) and not Dipendente.allInstances()->exists(d d.matricola = account.matricola) and not Supervisore.allInstances()->exists(s s.matricola = account.matricola)

3.3 TaskManagement

3.3.1 TaskCommonService

Nome Classe	TaskCommonService
Descrizione	espone le operazioni di lettura e le azioni condivise accessibili sia ai Dipendenti che ai Supervisor
Metodi	+ getTask(taskID: int) : Task + holdTask(taskID: int) : void + downloadAllegato(taskID) : Allegato
Invariante di classe	service:PersistenceService <> null
Firma Metodo	+ getTask(taskID: int) : Task
Descrizione	Recupera le informazioni complete di un task.
Pre-condizione	Context:TaskCommonService::getTask(taskID: int) taskID > 0

Post-condizione	Context:TaskCommonService::getTask(taskID: int) If Task.allInstances()->exists(t t.id = taskID) then results = Task.allInstances()->select(t t.is = TaskID) else result = null
Firma Metodo	+ holdTask(taskID: int) : boolean
Descrizione	Permette di cambiare sospendere un task e cambiare di conseguenza lo stato.
Pre-condizione	Context:TaskCommonService::holdTask(taskID: int) taskID > 0 and Task.allInstances()->select(t t.id = taskID).status = Tipi::stato::IN_ESECUZIONE or Tipi::stato::IN_SOSPENSIONE
Post-condizione	Context:TaskCommonService::holdTask(taskID: int) targetTask = Task.allInstances() -> select(t t.id = taskID); if targetTask <> null targetTask.stato = Tipi::stato::IN_SOSPENSIONE and result = true else result = false endif
Firma Metodo	+ downloadAllegato(taskID: int) : Allegato
Descrizione	Permette di scaricare un allegato dal task in cui è stato caricato.
Pre-condizione	Context:TaskCommonService::downloadAllegato(taskID: int) taskID > 0
Post-condizione	Context:TaskCommonService::downloadAllegato(taskID: int) targetAllegato = Allegato.allInstances() -> any(a a.taskID = taskID) If targetAllegato <> null result = targetAllegato else result = null endif

3.3.2 TaskSupervisoreService

Nome Classe	TaskSupervisoreService
Descrizione	espone tutte le operazioni eseguibili da un Supervisore sul ciclo di vita di un task
Metodi	+ getAllTaskSup(supervisore:Supervisore): List<Task> + createTask(titolo:String, istruzioni:String, dataScadenza:Date, priority: string, matricolaDip:String, matricolaSup:String, allegato: Allegato) : void + getAllDipendenteInfo(matricolaSup:int): List<Infomazione> + deleteTask(taskID: long) : bool + uploadAllegato(fileItem:FileItem, uploadDir:String) : Allegato
Invariante di classe	service:PersistenceService <> null
Firma Metodo	+ getAllTaskSup(supervisore:Supervisore): List<Task>
Descrizione	Permette di caricare tutte i task dal database assegnati dal supervisore inviato come parametro
Pre-condizione	context TaskManagementService::getAllTaskSup(supervisore:Supervisore) supervisore <> null and supervisore.ruolo=Tipi::ruolo::SUPERVISORE and supervisore.matricola > 0
Post-condizione	context TaskManagementService::getAllTaskSup(supervisore:Supervisore) tasksAttesi = Task.allInstances()->select(t t.supervisore = supervisore.matricola) result = tasksAttesi
Firma Metodo	+ createTask(titolo:String, dataCreazione:Date, dataScadenza:Date, istruzioni:String, stato:enum, matricolaDip:int, matricolaSup:int, priotita:enum, allegato: Allegato)
Descrizione	Permette di assegnare un task ad un dipendente.
Pre-condizione	Context taskSupervisoreService:: createTask(titolo:String, dataCreazione:Date, dataScadenza:Date, istruzioni:String, stato:enum, matricolaDip:int, matricolaSup:int, priotita:enum, allegato: Allegato)

	titolo <> null and titolo.size() > 0 and dataScadenza <> null and dataCreazione <> null and istruzioni <> null and istruzioni.size() >= 10 and stato <> null and matricolaDip > 0 and matricolaSup > 0 and priorita <> null and dataScadenza > dataCreazione and Dipendente.allInstances()->exists(d d.matricola = matricolaDip) and Supervisore.allInstances()-> exists(s s.matricola = matricolaSup)
Post-condizione	Context taskSupervisoreService:: createTask(titolo:String, istruzioni:String, dataScadenza:Date, priority: string, matricolaDip:String, matricolaSup:String, allegato: Allegato) Task.allInstances()->size() = Task.allInstances()@pre->size() + 1
Firma Metodo	+ getAllDipendenteInfo(supervisoreMat:int): List<Infomazione>
Descrizione	Permette di recuperare le informazioni di base dei dipendenti assegnati alla supervisore con matricola inviata come parametro
Pre-condizione	Context TaskSupervisoreService::getAllDipendenteInfo(supervisoreMat:int) supervisoreMat > 0
Post-condizione	Context TaskSupervisoreService::getAllDipendenteInfo(supervisoreMat:int) targetDipendenti = Dipendenti.allInstances()->select(d d.matricolaSup = supervisoreMat) result = targetDipendenti
Firma Metodo	+ deleteTask(taskID: int) : bool
Descrizione	Permette di rimuovere un task precedentemente assegnato dal sistema.
Pre-condizione	Context:TaskSupervisoreService:: deleteTask(taskID: int) taskID > 0
Post-condizione	Context:TaskSupervisoreService::deleteTask(taskID; int) targetTask = Task.allInstances() -> select(t t.id = taskID) if targetTask = null result = true

	<pre> else result = false endif </pre>
Firma Metodo	+ uploadAllegato(fileItem: FileItem, uploadDir: String) : Allegato
Descrizione	Permette di caricare all'interno del filesystem un file inviato dal supervisore durante la creazione di un task, restituisce Allegato per permettere di salvare le
Pre-condizione	<pre> context AllegatoService::uploadAllegato(fileItem: FileItem, uploadDir: String) fileItem <> null and fileItem.getName() <> null and uploadDir <> null </pre>
Post-condizione	<pre> context AllegatoService::uploadAllegato(fileItem: FileItem, uploadDir: String) result <> null and result.ocIsNew() </pre>

3.3.3 TaskDipendenteService

Nome Classe	TaskDipendenteService
Descrizione	espone tutte le operazioni eseguibili da un Dipendente sul ciclo di vita di un task.
Metodi	<pre> + getAllTaskDip(dipendente:Dipendente): List<Task> + inizializzaTask(taskID: int) : bool + completeTask(taskID: int) : bool </pre>
Invariante di classe	service:PersistenceService <> null
Firma Metodo	+ getAllTaskDip(dipendente:Dipendente): List<Task>
Descrizione	Permette di caricare tutti i task che sono stati assegnati al dipendente inviato come parametro.

Pre-condizione	Context TaskDipendenteService::getAllTaskDip(dipendente:Dipendente) Dipendente <>null and dipendente.matricola > 0 and dipendente.ruolo = Tipi::ruolo::DIPENDENTE
Post-condizione	Context TaskDipendenteService::getAllTaskDip(dipendente:Dipendente) tasksAttesi = Task.allInstances()->select(t t.dipendente =dipendente.matricola) result = tasksAttesi
Firma metodo	+ inizializzaTask(taskID: int) : bool
Descrizione	Segnala l'inizio dei lavori da parte di un dipendente, portando il task dallo stato "Da completare" a "In esecuzione".
Pre-condizione	Context:TaskDipendenteService::inizializzaTask(taskID: int) taskID > 0 and Task.allInstances()->select(t t.id = taskID).status <> Tipi::stato::COMPLETATO
Post-condizione	Context:TaskDipendenteService::inizializzaTask(taskID: int) targetTask = Task.allInstances() -> select(t t.id = taskID); if targetTask <> null targetTask.stato = Tipi::stato::IN_ESECUZIONE and result = true else result = false endif
Firma Metodo	+ completeTask(taskID: int) : bool
Descrizione	Segnala la conclusione del lavoro, portando il task nello stato finale "Completato".
Pre-condizione	Context:TaskDipendenteService::completeTask(taskID: int) taskID > 0 Task.allInstances()->select(t t.id = taskID).status = Tipi::stato::IN_ESECUZIONE
Post-condizione	Context:TaskDipendenteService::completeTask(taskID: int)

	<pre> targetTask = Task.allInstances() -> select(t t.id = taskID); if targetTask <> null targetTask.stato = Tipi::stato::COMPLETATO and result = true else result = false endif </pre>
--	---

3.4 NotificheManagement

3.4.1 NotificheService

Nome classe	NotificheService
Descrizione	Classe che serve per la gestione delle notifiche generate dai task del sistema e per la visualizzazione da parte del cliente
Metodi	<pre> +showNotification(utente:Utente):List<Notifica> +send(sender:int, receiver:int, message:String, type:enum):void </pre>
Invariante	\
Firma metodo	+showNotification(utente:Utente):List<Notifica>
Descrizione	Serve per recuperare le notifiche ricevute da parte di un utente
Pre-condizione	Context::NotificheService:showNotification(utente:Utente): List<Notifica> utente <> null and utente.matricola > 0
Post-condizione	NotificationController::showNotification(utente:Utente): List<Notifica> result = Notifica.allInstances-> select(n n.receiver = utente.matricola)
Firma metodo	+send(sender:int, receiver:int, message:String, type:enum):void

Descrizione	Implementa la logica di invio di una notifica. Crea una nuova istanza di notifica, la salva nel sistema persistente e tenta l'invio immediato al client del destinatario se connesso.
Pre-condizione	Context::NotificheService:send(senderMat:int, receiverMat:int, message:String, type:enum) sender > 0 AND receiver >0 AND message != null and type <> null
Post-condizione	Context::NotificheService:send(senderMat:int, receiverMat:int, message:String, type:enum) Notifica.allInstances()->exists(n n.senderMat = senderMat, n.receiverMat = receiverMat, n.message = message, n.type = type)

3.5 PersistenceManagement

3.5.1 PersistenceService

Nome classe	PersistenceService
Descrizione	Fornisce l'accesso ai DAO specifici per la gestione delle entità
Metodi	+ getUtenteDAO(): UtenteDAO + getTaskDAO(): TaskDAO + getNotificheDAO(): NotificheDAO
Invariante	\
Firma metodo	+ getUtenteDAO()
Descrizione	Restituisce l'istanza del DAO per la gestione degli utenti
Pre-condizione	\
Post-condizione	Context PersistenceService::getUtenteDAO() result.oclIsKindOf(UtenteDAO) AND result <> null
Firma metodo	+ getTaskDAO()
Descrizione	Restituisce l'istanza del DAO per la gestione dei task

Pre-condizione	\
Post-condizione	Context PersistenceService::getTaskDAO() result.oclIsKindOf(TaskDAO) AND result <> null
Firma metodo	+ getNotificheDAO()
Descrizione	Restituisce l'istanza del DAO per la gestione delle notifiche
Pre-condizione	\
Post-condizione	Context PersistenceService::getNotificheDAO() result.oclIsKindOf(NotificheDAO) and result <> null

Classi DAO

In questa sezione vengono specificate le interfacce per le classi DAO. Le classi DAO si occupano della gestione del database quindi i loro constraint punteranno a mantenere coerente il database sottostante, non conosceranno quindi la logica di business del sistema ma effettueranno controlli esclusivamente per assicurarsi un corretto inserimento all'interno delle tabelle dei dati passati dal service.

3.5.2 UtenteDAO

Nome classe	UtenteDAO
Descrizione	Gestisce la persistenza per l'entità Utente e le sue sottoclassi
Metodi	+ createUtente(utente:Utente, password:String) : void + createDipendente(dipendente:Dipendente, password:String) : void + createSupervisore(supervisore:Supervisore, password:String) : void - insertUtenteGenerico(connection:Connection, utente:Utente, password:String) : void + findByMatricola(matricola:int) : Utente + findByEmail(email:String) : Utente + getAllDipendentiInfo(matricola:int) : List<Informazioni> + getSupervisoreInfo(matricola:int) : Informazioni

	+ getAllSupervisor() : List<Informazioni> + getAllUtente(): List<Utente> + updateRuolo(matricola:int, nuovoRuolo:enum, matricolaNuovoSup:String) :void - deleteFromTable(connection:Connection, tabella:String, matricola:int) : void - insertIntoSupervisore(connection:Connection, matricola:int): void - insertIntoDipendente(connection:Connection, matricola:int, supMatricola:int):void + updateSupervisore(matricola:int, matricolaSup:int) : void + updatePassword(matricola:int, hashPassword:String) : void + delete(utente:Utente):void + recuperaPassword(email:String) : String
Invariante	\
Firma metodo	+createUtente(utente:Utente, password:String) : void
Descrizione	Persiste un nuovo utente di tipo Gestore nella tabella Utente del database
Pre-condizione	Context UtenteDAO::createUtente(utente:Utente, password:String) utente <> null and utente.ruolo = Tipi::ruolo::GESTORE and password <> null
Post-condizione	Context UtenteDAO::createUtente(utente:Utente, password:String) Utente.allInstances()->includes(utente)
Firma metodo	+ createDipendente(dipendente:Dipendente, password:String) : void
Descrizione	Permette di inserire un nuovo dipendente sia nella tabella Utente che nella tabella Dipendente del database
Pre-condizione	Context UtenteDAO:: createDipendente(dipendente:Dipendente, password:String) dipendente <> null and dipendente.ruolo =Tipi::ruolo::DIPENDENTE and password <> null and dipendente.supervisoreInfo <> null and dipendente.supervisoreInfo.matricola > 0 and Supervisore.allInstances()->exists(s s.matricola= dipendente.supervisoreInfo.matricola)

Post-condizione	Context UtenteDAO:: createDipendente(dipendente:Dipendente, password:String) Dipendente.allInstances()->includes(dipendente) Utente.allIstances()->includes(dipendente)
Firma metodo	createSupervisore(supervisore:Supervisore, password:String) : void
Descrizione	Permette di inserire un nuovo supervisore sia nella tabella Utente che nella tabella Supervisore del database
Pre-condizione	Context UtenteDAO:: ::createSupervisore(supervisore: Supervisore, password: String) supervisore <> null and supervisore.ruolo = Tipi::ruolo::SUPERVISORE password <> null
Post-condizione	Context UtenteDAO:: ::createSupervisore(supervisore: Supervisore, password: String) Supervisore.allInstances()->includes (supervisore) Utente.allIstances()->includes(supervisore)
Firma metodo	- insertUtenteGenerico(connection:Connection, utente:Utente, password:String) : void
Descrizione	Metodo di supporto per l'inserimento di nuovi utenti all'interno del database, viene utilizzato dai metodi createUtente, createDipendente, createSupervisore.
Pre-condizione	Context UtenteDAO:: insertUtenteGenerico(connection:Connection, utente:Utente, password:String) connection <> null and utente.email <> null and utente.email.size() <= 255 and utente.nome <> null and utente.nome.size()<= 50 and utente.cognome <> null and utente.cognome.size() <= 50 and utente.ruolo <> null and utente.dataNasc <> null and password <= 255 and not Utente.allIstances()->exists(u u.email = utente.email) [Constraint dovuto al campo email del db dichiarato come unique]
Post-condizione	Context UtenteDAO:: insertUtenteGenerico(connection:Connection, utente:Utente, password:String) Utente.allIstances()->includes(utente) Utente.allIstances()->size() = Utente.allIstances()->size()@pre + 1
Firma metodo	+ findByMatricola(matricola:int) : Utente
Descrizione	Recupera un utente tramite la sua matricola
Pre-condizione	Context UtenteDAO::findByMatricola(matricola:String)

	matricola > 0
Post-condizione	Context UtenteDAO::findByMatricola(matricola:String) targetUtente = Utente.allInstances()->select(u u.matricola = matricola) if targetUtente <> null result = targetUtente else result = null endif
Firma metodo	+ findByEmail(email:String) : Utente
Descrizione	Recupera dal database un Utente con email corrispondente al parametro passato
Pre-condizione	Context UtenteDAO::findByEmail(email: String) email <> null
Post-condizione	Context UtenteDAO::findByEmail(email: String) targetUtente = Utente.allInstances()->select(u u.email = email) if targetUtente <> null result = targetUtente else result = null endif
Firma metodo	+ getAllDipendentiInfo(matricola:int) : Informazioni:List
Descrizione	Recupera le informazioni di base (matricola, nome, cognome) di tutti i dipendenti di un supervisore la cui matricola è passata come parametro
Pre-condizione	Context UtenteDAO::getAllDipendentiInfo(matricola:int) matricola > 0
Post-condizione	Context UtenteDAO::getAllDipendentiInfo(matricola:int)

	<pre> result <> null and result->forAll(info Dipendente.allInstances()->select(d d.matricola = info.matricola and d.supMatricola = matricola)) </pre>
Firma metodo	+ getSupervisoreInfo(matricola:int) : Informazioni
Descrizione	Recupera le informazioni di base del supervisore assegnato al dipendente con matricola passata come parametro
Pre-condizione	Context UtenteDAO::getSupervisoreInfo(matricola:int) matricola > 0
Post-condizione	Context UtenteDAO::getSupervisoreInfo(matricola:int) targetDipendente = Dipendente.allInstances()->select(d d.matricola = matricola) if (targetDipendente <> null) result <> null and result = targetSupervisore else result = null
Firma metodo	+ getAllSupervisori() : List<Informazioni>
Descrizione	Permette di recuperare le informazioni principali di tutti i supervisori presenti nel sistema, utilizzata per inserire il form di inserimento di un nuovo dipendente
Pre-condizione	/
Post-condizione	Context UtenteDAO::getAllSupervisori() : List<Informazioni> Result = Utente.allInstances()->select (s s.ruolo = Tipi::Ruolo::SUPEVISORE)
Firma metodo	+ getAllUtente(): List<Utente>
Descrizione	Restituisce tutti gli utenti presenti all'interno del sistema
Pre-condizione	/
Post-condizione	Context UtenteDAO::getAllUtente() Result = Utente.allInstances()->select ()
Firma metodo	+ updateRuolo(matricola:int, nuovoRuolo:enum, matricolaNuovoSup:String) : void
Descrizione	Permette di aggiornare il ruolo di un utente, sia come colonna della tabella che poi come presenza nelle tabelle dipendente e supervisore

Pre-condizione	context UtenteDAO::updateRuolo(matricola: int, nuovoRuolo: Tipi::ruolo, matricolaNuovoSupervisore: String) matricola > 0 and UtenteDAO::getByMatricola(matricola) <> null nuovoRuolo <> and (nuovoRuolo = Tipi::ruolo::DIPENDENTE) implies (Supervisore.allInstances()->exists(s s.matricola = matricolaNuovoSupervisore) and matricola <> matricolaNuovoSupervisore)
Post-condizione	context UtenteDAO::updateRuolo(matricola: int, nuovoRuolo: Tipi::ruolo, matricolaNuovoSupervisore: String) Utente.allInstances()->exists(s s.matricola = matricola and s.ruolo = nuovoRuolo)
Firma metodo	- deleteFromTable(connection:Connection, tabella:String, matricola:int) : void
Descrizione	Metodo di supporto per mantenere la logica tra le tabella dopo aver effettuato un cambio di ruolo, cancella dal db la entry all'interno della tabella corrispondente
Pre-condizione	---
Post-condizione	Context UtenteDAO::deleteFromTable(connection:Connection, tabella:String, matricola:int) targetUtente = Utente.allInstances()->select(u u.matricola = matricola) if tabella = Supervisore not Supervisore.allInstances()->exists(s targetUtente.matricola = s.matricola) else not Dipendente.allInstances()->exists(s targetUtente.matricola = s.matricola)
Firma metodo	- insertIntoSupervisore(connection:Connection, matricola:int): void
Descrizione	Metodo di supporto per "updateRuolo" permette di inserire all'interno della tabella Supervisore una nuova entry
Pre-condizione	---
Post-condizione	Context UtenteDAO:: insertIntoSupervisore(connection:Connection, matricola:int) Supervisore.allInstances()->exists(s s.matricola = matricola)
Firma metodo	- insertIntoDipendente(connection:Connection, matricola:int, supMatricola:int) :void

Descrizione	Metodo di supporto per “updateRuolo” permette di inserire all’interno della tabella Dipendente una nuova entry
Pre-condizione	---
Post-condizione	Context UtenteDAO:: insertIntoDipendente(connection:Connection, matricola:int, supMatricola:int) Dipendente.allInstances()->exists(d d.matricola = matricola and d.matricolaSup = supMatricola)
Firma metodo	+ updateSupervisore(matricola:int, matricolaSup:int) : void
Descrizione	Serve a cambiare il supervisore assegnato ad un dipendente
Pre-condizione	Context UtenteDAO::updateSupervisore(matricola: int, matricolaSup: int) matricola > 0 and Dipendente.allInstances()->exists(d d.matricola = matricola) matricolaSup > 0 and Supervisore.allInstances()->exists(s s.matricola = matricolaSup) matricola <> matricolaSup
Post-condizione	Context UtenteDAO::updateSupervisore(matricola: int, matricolaSup: String) Dipendente.allInstances()->exists(d d.matricola = matricola and d.matricolaSup = matricolaSup)
Firma metodo	+ updatePassword(matricola:int, hashPassword:String) : void
Descrizione	Permette di sostituire la password salvata su database
Pre-condizione	context UtenteDAO::updatePassword(matricola:int, hashPassword: String) matricola > 0 and hashPassword <> null and hashPassword.size() <= 255
Post-condizione	context UtenteDAO::updatePassword(matricola: int, hashPassword: String) Utente.allInstances()->exists(u u.matricola = matricola and u.hashPassword = hashPassword)
Firma metodo	+ delete(utente:Utente):void
Descrizione	Elimina l’utente specificato dal database
Pre-condizione	Context UtenteDAO:: delete(utente:Utente) utente <> null and utente.matricola > 0 and not Dipendente.allInstances()->exists(d d.supMatricola = utente.matricola)

	[precondizione necessaria considerando l'integrità referenziale]
Post-condizione	Context UtenteDAO:: delete(utente:Utente) Utente.allInstances()->excludes (utente)
Firma metodo	+ recuperaPassword(email:String) : String
Descrizione	Permette di recuperare dal DB la password legata ad un email, utilizzata per fare il login al sistema
Pre-condizione	Context UtenteDAO::recuperaPassword(email:String) Email <> null
Post-condizione	Context UtenteDAO::recuperaPassword(email:String) targetUtente = UtenteDAO::findByEmail(email) if targetUtente <> null result = targetUtente.hashPassword else result = null;

3.5.2.1 TaskDAO

Nome classe	TaskDAO
Descrizione	Gestisce la persistenza per l'entità Task
Metodi	+ create (task:Task):void + findById(id:long) : Task + findByUtente(utente:Utente) : List<Task> + updateStatus(id:long, nuovoStato:enum) : void + update(task:Task) : void + delete (task:Task) : void
Invariante	\
Firma metodo	+ create (task:Task):void
Descrizione	Persistente una task all'interno del database
Pre-condizione	Context TaskDAO::create(task: Task) task <> null and

	task.titolo <> null and task.titolo.size() <= 50 and task.dataDiScadenza <> null and task.dataDiCreazione <> null and task.priorita <> null and task.istruzioni <> null and task.istruzioni.size() <= 2000 and task.stato <> null and task.supervisore <> null and Supervisore.allInstances()->exists(s s.matricola = task.supervisore) and task.dipendente <> null and Dipendente.allInstances()->exists(d d.matricola = task.dipendente)
Post-condizione	Context TaskDAO::create(task: Task) Task.allInstances()->includes(t) If task.allegato <> null Allegato.allInstances()->exists(a a.id = task.Allegato.id)
Firma metodo	+ findById(id:int) : Task
Descrizione	Trova un task in base al suo id
Pre-condizione	Context TaskDAO::findById(id:int) Id > 0
Post-condizione	Context TaskDAO::findById(id:int) targetTask = Task.allInstances()->select(t t.id = id) if Task.allInstances()-> exists(t t.id = id) result = targetTask else result = null
Firma metodo	+ findByUtente(utente:Utente) : List<Task>
Descrizione	Ritorna una lista di tutte i task assegnati ad un utente
Pre-condizione	Context TaskDAO::findByUtente(utente:Utente) utente <> null, utente.matricola > 0, utente.ruolo <> null and (utente.ruolo = Tipi::ruolo::DIPENDENTE or utente.ruolo = Tipi::ruolo::SUPERVISORE)
Post-condizione	Context TaskDAO::findByUtente(utente:Utente) If utente.ruolo = Tipi::ruolo::DIPENDENTE

	<pre> result = Task.allInstances()->select(t t.dipendente = utente.matricola) else if utente.ruolo = Tipi::ruolo::SUPERVISORE result = Task.allInstances()->select(t t.supervisore = utente.matricola) </pre>
Firma metodo	+ updateStatus(id:long, nuovoStato:enum) : void
Descrizione	Permette di cambiare lo stato di un task
Pre-condizione	<pre> context TaskDAO::updateStatus(id:long, nuovoStato:enum) nuovoStato <> null and id > 0 and Task.allInstances()->exists(t t.id = id) </pre>
Post-condizione	<pre> context TaskDAO::updateStatus(id:long, nuovoStato:enum) Task.allInstances()->exists(t t.id = id and t.stato = nuovoStato) </pre>
Firma metodo	+ delete (task:Task) : void
Descrizione	Elimina un task dal database
Pre-condizione	<pre> Context TaskDAO::delete(task: Task) task <> null and task.id > 0 </pre>
Post-condizione	<pre> Context TaskDAO::delete(task: Task) Task.allInstances()->excludes (t) AND Allegato.allInstances()->excludes (a a.taskId <> t.id) </pre>

3.5.3 NotificheDAO

Nome classe	NotificaDAO
Descrizione	Gestisce la persistenza per l'entità Notifica
Metodi	<pre> + create(notifica:Notifica):void + findByUtente(utente:Utente): List<Notifica> </pre>

Invariante	\
Firma metodo	+ create(notifica:Notifica):void
Descrizione	Inserisce all'interno del database una notifica
Pre-condizione	Context NotificaDAO::create(notifica: Notifica) notifica <> null
Post-condizione	Context NotificaDAO::create(notifica: Notifica) Notifica.allInstances()->includes(notifica)
Firma metodo	+ findByUtente(utente:Utente): List<Notifica>
Descrizione	Carica dal database tutte le notifiche dell'utente specificato
Pre-condizione	Context NotificaDAO::findByUtente(utente: Utente) utente <> null
Post-condizione	Context NotificaDAO::findByUtente(utente: Utente) result = Notifica.allInstances()->select(n n.receiver = utente.email)